

Universidad del Valle de Guatemala

Facultad de Ingeniería

Programación Orientada a Objetos

Catedrático: Erick Marroquín



Sistema de Control de Órdenes de Servicio de Un Taller Automotriz

Jared Eduardo Guerra Aguirre

Carné: 26502

Ciencia de la Computación y Tecnologías de la Información

Sección 11

25 de agosto de 2026

1. Requisitos funcionales

Almacenar los datos de una orden: El sistema deberá representar cada orden mediante un objeto que almacene el número de orden, nombre del propietario, placa del vehículo, descripción del servicio y costo estimado. Estos datos estarán encapsulados dentro de la clase OrdenServicio.

Administrar una cantidad dinámica de órdenes: El sistema deberá guardar una cantidad variable de órdenes, sin un límite fijo. Se utilizará una colección dinámica que aumentará o disminuirá de tamaño conforme se registren o cancelen órdenes.

Utilizar List y ArrayList: La colección se declarará utilizando la interfaz List<OrdenServicio> y se inicializará con una instancia de ArrayList<OrdenServicio>. Esto permite trabajar con una abstracción y utilizar una implementación dinámica concreta.

Validar el registro de órdenes: Antes de registrar una orden, el sistema deberá comprobar que su número no se encuentre repetido, que el nombre, la placa y la descripción no estén vacíos, y que el costo estimado sea mayor que cero. Una orden que incumpla alguna condición no será agregada.

Consultar todas las órdenes: El sistema deberá permitir recorrer y mostrar todas las órdenes almacenadas. Si la colección está vacía, deberá informar que no existen órdenes registradas.

Buscar una orden por número: El usuario podrá proporcionar un número de orden para localizar una orden específica. La búsqueda recorrerá la colección y comparará el número solicitado con el número de cada objeto.

Controlar búsquedas sin resultados: Cuando la orden exista, el sistema mostrará toda su información. Si no se encuentra, se lanzará y capturará una OrdenNoEncontradaException, informando el problema sin finalizar la ejecución del programa.

Modificar una orden: El sistema deberá permitir cambiar la descripción del servicio y el costo estimado de una orden existente. La nueva descripción no podrá estar vacía y el nuevo costo deberá ser mayor que cero.

Cancelar una orden: El sistema deberá permitir cancelar una orden utilizando su número. La cancelación eliminará de la colección el objeto correspondiente, si la orden no existe, será controlado con una excepción.

Consultar órdenes por placa: El sistema deberá buscar todas las órdenes cuya placa coincida con la ingresada por el usuario. El resultado será una lista porque un mismo vehículo puede poseer varias órdenes registradas.

Calcular el costo total: El sistema deberá recorrer las órdenes activas y sumar sus costos estimados. Debido a que cancelar una orden la elimina de la colección, todos los objetos que estén almacenados se considerarán órdenes activas.

Calcular el costo promedio: El sistema calculará el promedio dividiendo el costo total entre la cantidad de órdenes activas. Si no existen órdenes, devolverá 0.0 para evitar una división entre cero.

Determinar la orden de mayor costo: El sistema deberá recorrer la colección y comparar los costos para identificar la orden con el costo estimado más alto. Si la colección está vacía se mostrará que no existe una orden que pueda evaluarse.

Contar las órdenes registradas: El sistema deberá mostrar la cantidad actual de órdenes utilizando el tamaño de la colección dinámica. El resultado cambiará automáticamente cuando una orden sea registrada o cancelada.

Controlar entradas incorrectas: Las conversiones de datos ingresados por el usuario serán protegidas usando try-catch. Por ejemplo, se capturará NumberFormatException cuando se ingrese texto en un campo que requiere un número entero o decimal.

Controlar operaciones sobre órdenes inexistentes: Las operaciones de búsqueda, modificación y cancelación usarán try-catch para controlar la excepción producida cuando el número solicitado no corresponda a ninguna orden registrada.

Utilizar un bloque finally: Por lo menos una operación de búsqueda, modificación o cancelación tendrá un bloque finally. Este bloque mostrará un mensaje como “Operación finalizada, regresando al menú principal”, independientemente de que la operación haya tenido éxito o haya producido una excepción.

Mantener el programa en ejecución: Después de capturar una excepción, el sistema deberá regresar al menú principal. El ciclo del menú solamente terminará cuando el usuario seleccione la opción salir.

2. Propósito de las clases

Clase OrdenServicio: Representar una orden de servicio individual. Tiene los datos del propietario, del vehículo y del servicio solicitado. Mantiene estos datos encapsulados y valida las modificaciones realizadas sobre la descripción y el costo.

Clase GestorOrdenes: Administrar las órdenes de servicio activas con una colección dinámica. Tiene la lógica para registrar, consultar, buscar, modificar y cancelar órdenes, además de calcular los reportes solicitados.

Clase OrdenNoEncontradaException: Representar una situación excepcional en la que se intenta buscar, modificar o cancelar una orden cuyo número no está registrado. Permite separar este error de otros tipos de entradas inválidas.

Clase Main: Es el driver program del sistema. Presenta el menú, recibe los datos ingresados por el usuario, llama a los métodos de GestorOrdenes, muestra los resultados y controla las excepciones para evitar que el programa finalice inesperadamente.

1. Tabla de atributos clase OrdenServicio

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
OrdenServicio	numeroOrden	int	private	Identificar de manera única una orden.
OrdenServicio	nombrePropietario	String	private	Guardar el nombre del propietario.
OrdenServicio	placaVehiculo	String	private	Guardar la placa del vehículo relacionado con la orden.
OrdenServicio	descripcionServicio	String	private	Describir el mantenimiento o reparación necesaria
OrdenServicio	costoEstimado	double	private	Guardar el costo estimado del servicio

2. Tabla de métodos clase OrdenServicio

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
OrdenServicio	OrdenServicio	-	public	int numeroOrden, String nombrePropietario, String placaVehiculo, String descripcionServicio, double costoEstimado	Crear una orden con sus valores iniciales. Lanza IllegalArgumentException si un texto está vacío o el costo no es mayor que cero.
OrdenServicio	getNumeroOrden	int	public	-	Obtener el número de la orden.
OrdenServicio	getNombrePropietario	String	public	-	Obtener el nombre del propietario.
OrdenServicio	getPlacaVehiculo	String	public	-	Obtener la placa del vehículo.
OrdenServicio	getDescripcionServicio	String	public	-	Obtener la descripción del servicio.
OrdenServicio	getCostoEstimado	double	public	-	Obtener el costo estimado.
OrdenServicio	setDescripcionServicio	void	public	String descripcionServicio	Modificar la descripción. Lanza IllegalArgumentException si el nuevo valor está vacío.
OrdenServicio	setCostoEstimado	void	public	double costoEstimado	Modificar el costo estimado. Lanza IllegalArgumentException si el nuevo costo es menor o igual que cero.
OrdenServicio	toString	String	public	-	Devolver una cadena con toda la información de la orden.
OrdenServicio	validarTexto	void	private	String valor, String nombreCampo	Comprobar que un texto no sea nulo ni esté vacío. Lanza IllegalArgumentException cuando el dato no es válido.

3. Tabla de atributos clase GestorOrdenes

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
GestorOrdenes	ordenes	List<OrdenServicio>	private	Guardar todas las ordenes activas de forma dinámica. Se inicializa con new ArrayList<>()

4. Tabla de métodos clase GestorOrdenes

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
GestorOrdenes	GestorOrdenes	-	public	-	Inicializar la colección con new ArrayList<>().
GestorOrdenes	registrarOrden	void	public	OrdenServicio orden	Agregar una orden a la colección. Lanza IllegalArgumentException si el objeto es nulo o su número ya está registrado.
GestorOrdenes	consultarOrdenes	List<OrdenServicio>	public	-	Devolver una copia de las órdenes registradas para permitir su consulta sin exponer directamente la colección interna.
GestorOrdenes	buscarOrden	OrdenServicio	public	int numeroOrden	Buscar y devolver una orden por su número. Lanza OrdenNoEncontradaException si no existe.
GestorOrdenes	modificarOrden	void	public	int numeroOrden, String nuevaDescripcion, double nuevoCosto	Buscar una orden y modificar su descripción y costo. Lanza OrdenNoEncontradaException si no existe e IllegalArgumentException si los nuevos datos son inválidos.
GestorOrdenes	cancelarOrden	void	public	int numeroOrden	Buscar y eliminar una orden de la colección. Lanza OrdenNoEncontradaException si la orden no existe.
GestorOrdenes	buscarPorPlaca	List<OrdenServicio>	public	String placa	Devolver una lista con todas las órdenes asociadas a la placa. Lanza IllegalArgumentException si la placa está vacía.
GestorOrdenes	calcularCostoTotal	double	public	-	Sumar y devolver los costos estimados de todas las órdenes activas.
GestorOrdenes	calcularCostoPromedio	double	public	-	Calcular y devolver el costo promedio. Devuelve 0.0 cuando no existen órdenes para evitar una división entre cero.
GestorOrdenes	obtenerOrdenMayorCosto	OrdenServicio	public	-	Buscar y devolver la orden de mayor costo. Lanza OrdenNoEncontradaException si la colección está vacía.
GestorOrdenes	obtenerCantidadOrdenes	int	public	-	Devolver la cantidad actual de órdenes mediante el tamaño de la colección.
GestorOrdenes	existeNumeroOrden	boolean	private	int numeroOrden	Comprobar si un número de orden ya se encuentra registrado.

5. Tabla de atributos clase OrdenNoEncontradaException

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
OrdenNoEncontradaException	-	-	-	No necesita atributos propios, usa el mensaje heredado de Exception

6. Tabla de métodos clase OrdenNoEncontradaException

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
OrdenNoEncontradaException	OrdenNoEncontradaException	-	public	String mensaje	Crear una excepción con un mensaje que explique que la orden solicitada no fue encontrada.

7. Tabla de atributos clase Main

Clase	Atributo	Tipo de dato	Visibilidad	Propósito
Main	entrada	Scanner	private static final	Leer los datos ingresados por el usuario.
Main	gestor	GestorOrdenes	private static final	Permitir que el programa principal administre las órdenes.

8. Tabla de métodos clase Main

Clase	Método	Retorno	Visibilidad	Parámetros	Propósito
Main	main	void	public static	String[] args	Iniciar el programa, mantener activo el ciclo del menú y finalizarlo únicamente cuando el usuario seleccione salir.
Main	mostrarMenu	void	private static	-	Mostrar las diez opciones disponibles en el sistema.
Main	registrarOrden	void	private static	-	Solicitar los datos y registrar una orden. Captura IllegalArgumentException si los datos son inválidos o el número está repetido.
Main	consultarOrdenes	void	private static	-	Obtener y mostrar las órdenes registradas o informar que la colección está vacía.
Main	buscarOrden	void	private static	-	Solicitar el número y mostrar la orden. Captura OrdenNoEncontradaException si no existe y ejecuta un bloque finally al terminar.
Main	modificarOrden	void	private static	-	Solicitar los nuevos datos de una orden. Captura OrdenNoEncontradaException si no existe e IllegalArgumentException si los datos son inválidos.
Main	cancelarOrden	void	private static	-	Solicitar un número y cancelar la orden. Captura OrdenNoEncontradaException si la orden no existe.
Main	consultarPorPlaca	void	private static	-	Solicitar una placa y mostrar todas las órdenes asociadas. Controla una placa vacía e informa si no hay coincidencias.
Main	mostrarReporteCostos	void	private static	-	Mostrar el costo total y el costo promedio de las órdenes activas.

Main	mostrarOrdenMayorCosto	void	private static	-	Mostrar la orden con mayor costo. Captura OrdenNoEncontradaException si la colección está vacía.
Main	mostrarCantidadOrdenes	void	private static	-	Mostrar la cantidad actual de órdenes registradas.
Main	leerEntero	int	private static	String mensaje	Solicitar y convertir un número entero. Captura NumberFormatException y vuelve a solicitar el dato si la entrada no es válida.
Main	leerDouble	double	private static	String mensaje	Solicitar y convertir un número decimal. Captura NumberFormatException y vuelve a solicitar el dato si la entrada no es válida.
Main	leerTextoNoVacio	String	private static	String mensaje	Solicitar texto y repetir la lectura hasta que el usuario ingrese un valor que no esté vacío.

9. Diagrama de clases

